



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта

Кафедра высшей математики

КУРСОВАЯ РАБОТА
по дисциплине
«Объектно-ориентированное программирование»

Тема курсовой работы
«Моделирование декартова дерева по неявному ключу»

Студентка группы КМБО-01-22

Ахаминова Л.З.

Руководитель курсовой работы
доцент кафедры Высшей математики
к.ф.-м.н.

Петрусеви́ч Д.А.

Работа представлена к
защите

«21» *дека* 20 *23* г.

Л.З. Ахаминова
(подпись студентки)

«Допущена к защите»

«21» *дека* 20 *23* г.

Д.А. Петрусеви́ч
(подпись руководителя)



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта

Кафедра высшей математики

Утверждаю
Исполняющий обязанности заведующего
кафедрой *А.В.Шатина* А.В.Шатина

«22» сентября 2023 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Объектно-ориентированное программирование»

Студентка *Ахаминова Л.З.*

Группа *КМБО-01-22*

1. Тема: «Моделирование декартова дерева по неявному ключу»
2. Исходные данные:

Реализовать класс «Декартово дерево по неявному ключу»

Реализовать необходимые для класса функции: конструкторы, деструкторы, нужные методы (вставка, удаление элемента, merge, split)

Реализовать поиск значения допустимой для дерева функции на отрезке (сумма, максимум, переворот)

Реализовать групповое изменение значений в определенном отрезке

Для выполнения групповых операций реализовать отложенное исполнение команд

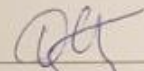
Решить задачи в двух вариантах: 1) быстром (в результате операций на деревьях получаем новое дерево, старыми не пользуемся), 2) безопасном (результатом операции на деревьях становится новое дерево, операнды не меняются и готовы к дальнейшему применению)

3. Перечень вопросов, подлежащих разработке, и обязательного графического материала:
Продемонстрировать работу декартова дерева на произвольных отрезках и при различных групповых функциях

4. Срок представления к защите курсовой работы: до «22» декабря 2023 г.

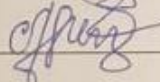
Задание на курсовую
работу выдал

«22» сентября 2023 г.

 (Петрусеви́ч Д.А.)

Задание на курсовую
работу получила

«22» сентября 2023 г.

 (Ахаминова Л.С.)

Оглавление

Введение	3
Глава 1. Разбор алгоритма	4
1.1 Теоритические сведения	4
1.2 Реализация и описание операций	5
1.3 Декартово дерево по неявному ключу	7
1.4 Сравнение с другими структурами	8
Глава 2. Описание программы.....	11
2.1 Реализация	11
2.2 Основные операции.....	12
2.3 Декартово дерево по неявному ключу	13
2.4 Демонстрация работы в main()	14
Заключение	16
Список литературы	17
Приложение	18

Введение

Декартово дерево – красивая и непростая структура данных, которая позволяет производить многие операции над массивами данных за логарифмическое время. В массиве каждый элемент имеет какой-то ключ. Эти ключи задают на этом массиве определенный порядок, и все запросы к дереву обычно каким-то образом привязаны к этому порядку. И это прекрасно работает, пока нам не требуются запросы, которые этот порядок как-то нетривиально меняют. Но когда появляются такие функции, которым нужно уметь выводить сумму на произвольном отрезке и «переворачивать» произвольный отрезок, на помощь приходит декартово дерево, неявное по ключу. В англоязычной литературе данную структуру обычно называют treap от слияния слов tree(дерево) и heap(куча/пирамида). Попытки дать похожее название декартову дереву в русском языке не ограничились одним вариантом из-за неоднозначного перевода слова heap. Поэтому можно встретить такие названия как дерево + пирамида = дерамида, куча + дерево = курево, дерево + куча = дуча, пирамида + дерево = пиво.

В данной работе описан и разобран алгоритм работы декартова дерева по неявному ключу. Сравнены похожие алгоритмы, время работы, объем занимаемой памяти.

Работа разделена на две главы. В первой приведена теория и сравнение. Во второй более подробно разобрана реализация с точки зрения написания кода.

Глава 1. Разбор алгоритма

1.1 Теоритические сведения

Для того чтобы разобраться с понятием декартова дерева, неявного по ключу, необходимо сначала пояснить некоторые термины.

Бинарное дерево поиска (англ. *binary search tree*, *BST*) — структура данных для работы с упорядоченными множествами, обладающая следующим свойством: если x — узел бинарного дерева с ключом k , то все узлы в левом поддереве должны иметь ключи, меньшие k , а в правом поддереве большие k .

Бинарная куча (англ. *binary heap*) — реализация очереди с приоритетами, использующая корневое дерево, для которого выполнены три условия:

1. Значение в любой вершине не больше, чем значения её потомков.
2. У любой вершины не более двух детей.
3. Слои заполняются последовательно сверху вниз и слева направо, без «дырок».

Декартово дерево (англ. *treap*) - это структура данных, объединяющая в себе бинарное дерево поиска и бинарную кучу. Деревьев поиска с одним и тем же набором элементов может быть много. Наличие приоритетов сокращает все возможности до одного дерева. По сути, декартово дерево является одним из типов балансирующих деревьев. Например, АВЛ или красно-черные деревья также имеют возможность балансировки, однако в них нужно запоминать частные случаи преобразований (поворотов) для балансировки, в то время как в декартовых деревьях все преобразования сводятся к двум операциям. Графически их легко представить в виде графика, где по оси x откладываются ключи, а по оси y – приоритеты (рис 1).

Подобная структура данных была предложена Сиделем (Siedel) и Арагоном (Aragon) в 1989 г.

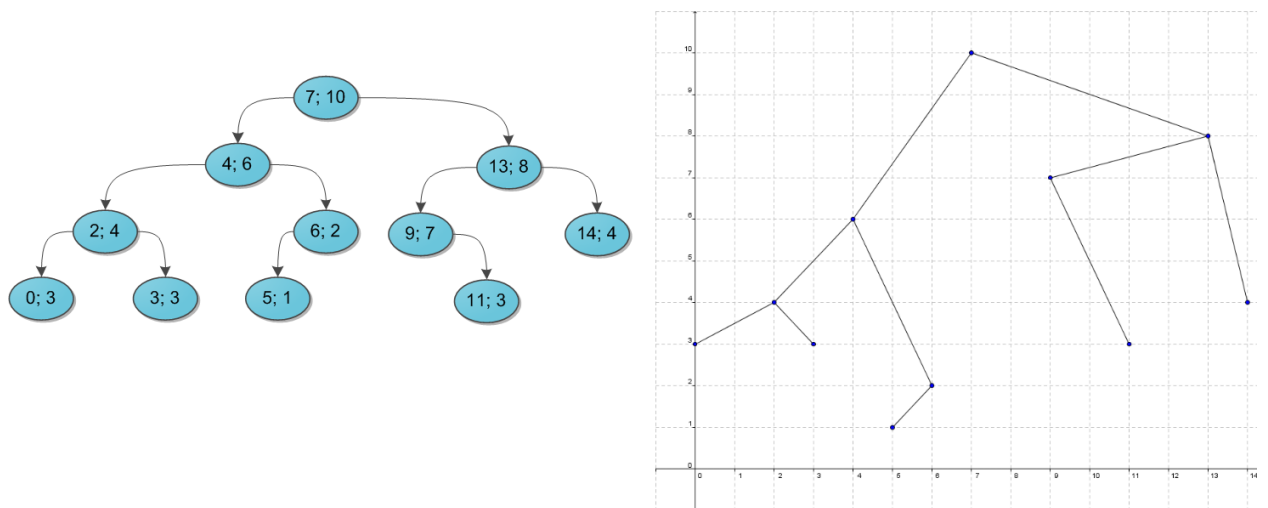


Рис 1. Декартово дерево

Более строго это структура данных, которая хранит пары (X, Y) («ключ», «приоритет») в виде бинарного дерева таким образом, что она является бинарным деревом поиска по x (меньше – левее, больше – правее) и бинарной кучей по y (меньше – ниже, больше – выше). Предполагая, что все X и все Y являются различными, получаем, что если некоторый элемент дерева содержит (X_0, Y_0) , то у всех элементов в левом поддереве $X < X_0$, у всех элементов в правом поддереве $X > X_0$, а также и в левом, и в правом поддереве имеем: $Y < Y_0$. X здесь являются ключами, а Y приоритетами. Из этого также следует очевидный вывод: дерево строится однозначно.

С небольшими модификациями, декартово дерево умеет всё то же, что и любое бинарное дерево поиска, например:

- добавить число x во множество;
- определить, есть ли во множестве число x ;
- найти первое число, не меньшее x ;
- найти количество чисел в промежутке $[l, r]$.

Как и во всех сбалансированных деревьях поиска, все операции работают за высоту дерева: $O(\log n)$.

Декартово дерево неявное по ключу отличается от обычного декартова дерева тем, что в нем нет ключей. Вместо этого мы добавляем индексацию итераций, тем самым присваивая каждому узлу уникальный номер.

1.2 Реализация и описание операций

Информацию о каждом узле удобно хранить в отдельном классе. Создадим его и назовем `Node`. В закрытых переменных будем хранить ключи, приоритеты, указатели на правого и левого ребенка, также храним в узле значение функции над элементами массива. В листьях – значение функции на элементе. В родительских элементах – значение на отрезке массива

(предполагается, что его можно быстро пересчитать при изменении массива). В разделе public опишем работу конструкторов, функции вывода, доступа к закрытым переменным, некоторых вспомогательных функций и т. д. Далее создадим класс дерево. В нем необходимо описать 2 главные функции, которые позволят реализовать любую операцию на дереве. Речь идет о Merge и Split.

Merge (рис. 2) – операция, которая объединяет два поддерева и возвращает получившееся новое дерево. Для ее работы требуется, чтобы все ключи одного дерева были меньше, чем все ключи другого. Корнем в новом дереве становится один из двух корней, т. е. элемент с наибольшим приоритетом. Эта операция реализуется за $O(\log N)$. Очевидно, что в ней мы должны использовать рекурсию.

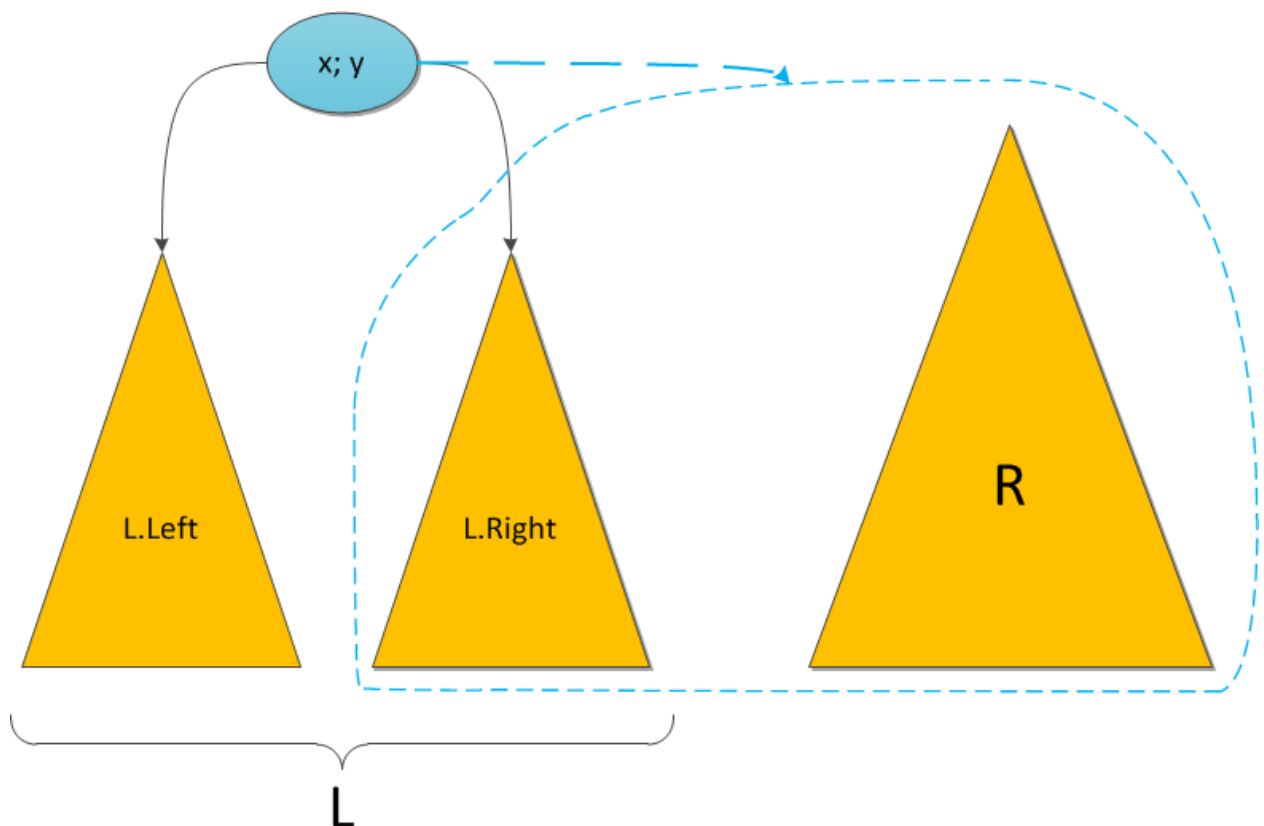


Рис 2. Операция Merge

Split (рис. 3) – операция, которая разрезает дерево и возвращает два новых. Алгоритм ее работы заключается в разбиении дерева на два таким образом, чтобы в первом ключи были меньше x (Left), а во втором - больше (Right). Результатом разрезания становятся: два дерева, вырезанный элемент с ключом x (если он есть). Корень дерева отправляется налево, если ключ корня меньше x или направо, если ключ корня больше x .

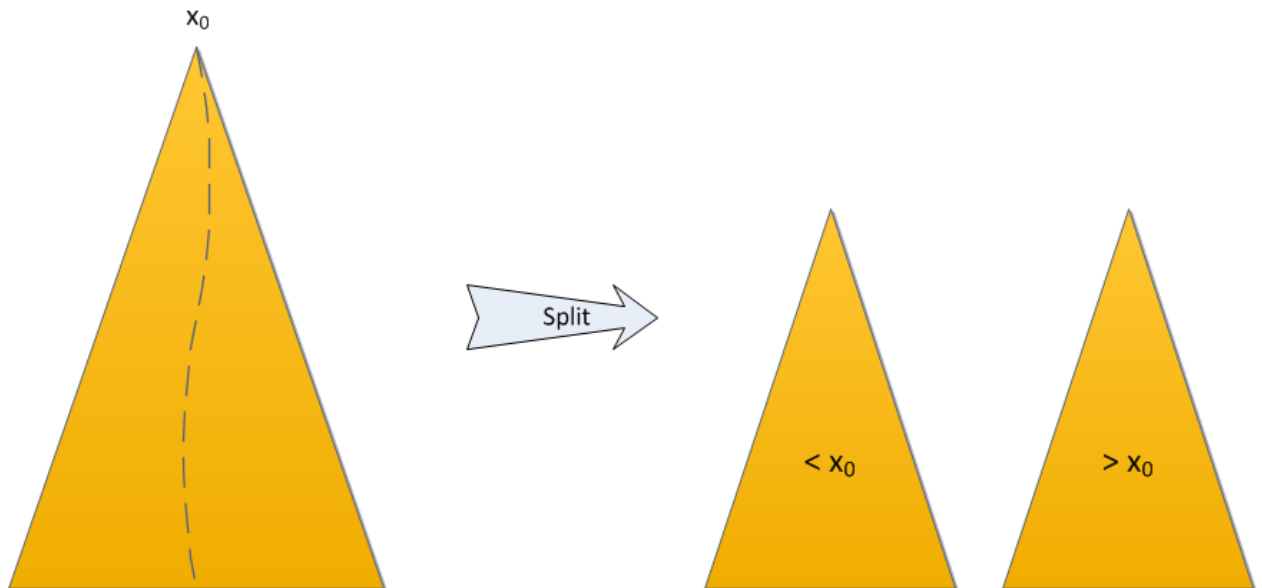


Рис 3. Операция Split

В книге Кормена [1] приводится доказательство того, что при выполнении большого количества операций `merge()` / `split()` высота дерева сходится по вероятности к величине, пропорциональной $\log n$, а вместе с ней и `merge()` / `split()` оцениваются за $O(\log n)$.

Теперь все операции можно выразить на основе Merge и Split.

Вставка (Insert x)

- Разделим (Split) дерево по ключу x на дерево L , с ключами меньше x , и дерево R , с БОльшими.
- Создадим из данного ключа дерево M из единственной вершины (x, y) , где y — только что сгенерированный случайный приоритет.
- Объединим (Merge) по очереди L с M , то, что получилось — с R .

Удаление (Remove x):

- Разделить (Split) дерево по ключу $x-1$ на L и R
- Разделить (Split) R по ключу x на $R.L$, $R.R$
- Объединить (Merge) L , $R.L$, $R.R$

Также добавим в узлы некоторые полезные переменные. Например, `size` — число дочерних элементов. На его основе построим индекс/номер итерации, на которой пройден элемент в InOrder-обходе.

1.3 Декартово дерево по неявному ключу

Декартово дерево, неявное по ключу (рис. 4) - это простая модификация обычного декартового дерева. Ключевая идея заключается в том, что в качестве ключей следует неявно использовать индексы элементов в InOrder-обходе. Другими словами это простое представление массива в виде дерева,

которое позволяет производить с ним и с его подмассивами некоторые операций за логарифмическое время.

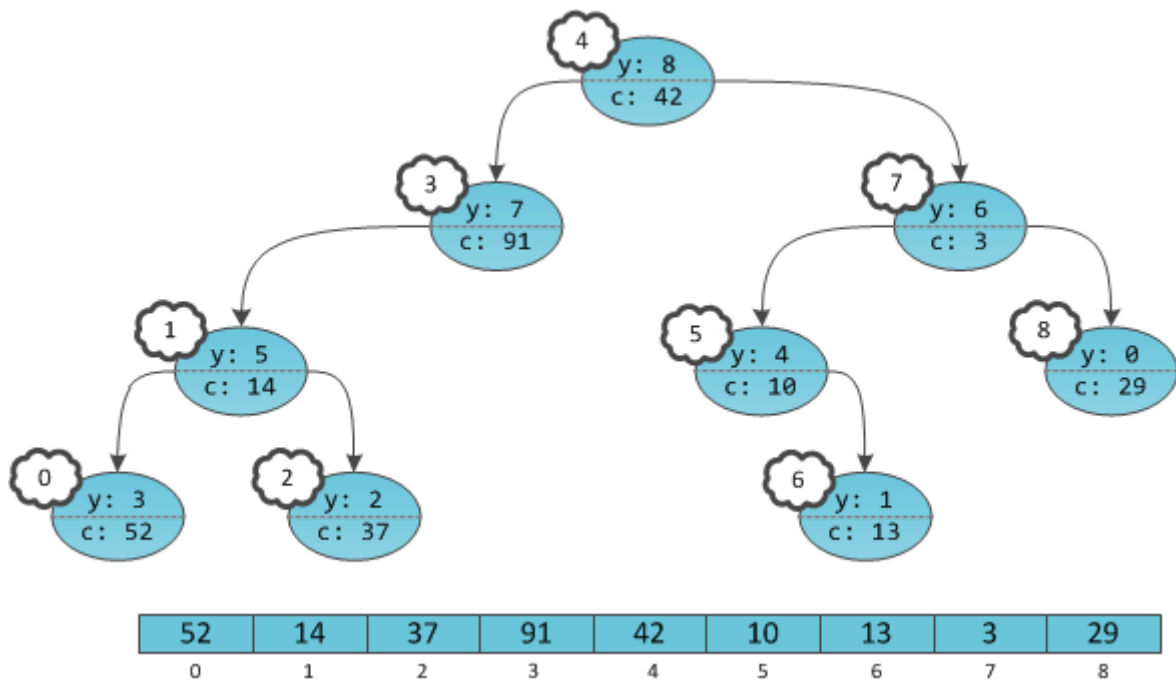


Рис 4. Декартово дерево по неявному ключу

Создадим новый класс `ImplicitTree` для декартова дерева по неявному ключу. В программе будем наследовать декартово дерево, т. к. многие операции идентичны. Например, `Merge` совсем не использует ключи, предполагая, что $L.key < R.key$. Сбалансированным дерево останется из-за приоритетов. Однако функция `Split` работает с ключами, поэтому в новом классе необходимо будет переопределить данную функцию. Изменением будет то, что теперь вместо ключей мы будем использовать индекс в массиве (`size` в дереве)

Можно разнообразить класс, добавив функции, которые помогут раскрыть возможности и удобства использования именно декартовых деревьев, неявных по ключу, а не, например, массивов.

Главными отличительными чертами является то, что мы можем переставлять целые диапазоны элементов в разные части массива, а также совершать различные операции на любом месте и с любым количеством элементов за относительно маленькое время. Это открывает много возможностей. Например, теперь доступно переворачивать массив. С обычным декартовым деревом это не сработало бы, так как подобное действие нарушило бы структуру декартова дерева. Но ввиду отсутствия ключей, подобные ограничения отпадают.

1.4 Сравнение с другими структурами

Другие похожие структуры (например, дерево отрезков, дерево Фенвика) отличаются от нашего дерева реализацией, скоростью выполнения некоторых функций, размером используемой памяти. Декартово дерево,

неявное по ключу, позволяет производить операции над массивом за логарифмическое время. Однако оно занимает больше памяти, чем другие структуры. При небольшом объеме памяти лучше использовать дерево отрезков. Но в нем круг операций, поддерживаемых на отрезке массива, уже, чем у декартова дерева. Самую простую реализацию и минимальные затраты по памяти имеет дерево Фенвика. Это простая структура данных, но круг операций, поддерживаемых на отрезке массива, у нее самый узкий.

Подробное сравнение с операциями других структур приведено в Таблице 1.

Таблица 1. Сравнение, сложность операций

	высота	Max	поиск	Merge	Insert/Remove
Куча	$\log n$	$O(1)$	$O(\log n)$	$O(n \log n)$	$O(\log n)$
Дерево поиска	$\log n$	$O(1)$	$O(\log n)$	$O(m+n)$	$O(\log n)$
Декартово дерево	$4 \log n$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Декартово дерево, неявное по ключу	$4 \log n$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Дерево отрезков	$\log n$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Дерево Фенвика	$4 \log n$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

У таких структур как декартово дерево, дерево отрезков, декартово дерево по неявному ключу и дерево отрезков есть групповые операции. Несмотря на то, что требования к групповым функциям в этих структурах данных имеют некоторые сходства, они также имеют свои особенности, связанные с порядком обхода элементов и операциями структуры данных.

У декартова дерева групповая функция должна быть ассоциативной и коммутативной, так как порядок обхода элементов может быть произвольным. Также она должна быть устойчивой к изменению порядка элементов, так как порядок элементов может меняться при поворотах дерева.

В декартовом дереве по неявному ключу групповая функция должна быть ассоциативной и коммутативной, так как порядок обхода элементов может быть произвольным. Также она должна быть устойчивой к изменению порядка элементов, так как порядок элементов может меняться при операциях Split и Merge.

В дерево Фенвика групповая функция должна быть ассоциативной и обратимой, так как операции обновления и запроса могут быть произведены в любом порядке. Она должна иметь нейтральный элемент, так как это используется при обновлении и запросе.

В дереве отрезков групповая функция должна быть ассоциативной и коммутативной, так как порядок обхода элементов может быть произвольным. Она должна быть устойчивой к изменению порядка элементов, так как порядок элементов может меняться при операциях объединения отрезков.

Глава 2. Описание программы

2.1 Реализация

В начале кода сразу же необходимо ввести класс для узлов дерева. В полях `private` будут храниться переменные ключ, приоритет, число дочерних элементов, переменные *max* для корректной работы групповых операций. А также указатели на правого и левого ребенка. Код представлен в Код 1.

```
1. class node {  
2. protected:  
3. K key;  
4. T value;  
5. int size;  
6. D product;  
7. node<K, T, D>* left;  
8. node<K, T, D>* right;  
9. node<K, T, D>* parent;  
10. D data;
```

Код 1. Доступ – `private` в классе узел

Вначале доступной пользователю области находятся конструкторы класса, устанавливающие стартовые значения для указателей, функции для доступа к переменным и установки их значений с помощью обычных геттеров и сеттеров, также напомним функцию `successor` для поиска следующего элемента по ключу (данная операция применяется в функции `Split`). После дружественные функции вывода данных о дереве. Код представлен в Код 2.

```
1. node<K, T, D>* succs() {  
2. if (right == NULL) {  
3. node<K, T, D>* tmp = this;  
4. while (true) {  
5. if (tmp->parent == NULL)  
6. return tmp->parent;  
7. else if (tmp->parent->key > key) { return tmp->parent; }  
8. else { tmp = tmp->parent; }  
9. }  
10. }  
11. else {  
12. node<K, T, D>* tmp = this->right;  
13. while (tmp->left != NULL) { tmp = tmp->left; }  
14. return tmp;  
15. }  
16. }
```

Код 2. Функции для доступа к полям класса извне

Следующий класс – класс итератор для перемещения по узлам дерева, который требует множество небольших вспомогательных функций, в том числе и булевых для сравнений. Теперь можно уже создать основной класс

декартова дерева. Т. к. дерево по неявному ключу практически идентично обычному декартову дереву, имеет смысл реализовать сначала класс декартова дерева, после наследовать от него основные функции и т. д. в дерево по неявному ключу. В полях `private` этого класса достаточно хранить указатель на вершину. Код представлен в Код 3.

```
1. class treap {
2. protected:
3.     node<K, T, D>* root;
```

Код 3. Класс Декартово дерево

2.2 Основные операции

Одной из двух важнейших операций, на которых строятся почти все остальные функции, является Merge. Сначала делаем проверку на пустой корень. Далее, проверяя приоритеты и не забывая следить за пустыми указателями, рекурсивно проходим по дереву. После можно сделать еще две перегрузки функции Merge для присоединения отдельного узла к дереву. Код представлен в Код 5. Более подробный код приведен в Приложении.

```
1. node<K, T, D>* merge(node<K, T, D>* L, node<K, T, D>* R) {
2.     if (L == NULL || R == NULL) return (L) ? L : R;
3.     if (L->getvalue() > R->getvalue()) {
4.         if (L->getkey() < R->getkey()) { ... }
5.         else { ... }
6.         return L;
7.     }
8.     else {
9.         if (L->getkey() < R->getkey()) { ... }
10.        else { ... }
11.        return R;
12.    }
13. }
```

Код 5. Функция Merge

Следующая важная операция – Split.

Выводить результат данная функция будет в два поддерева. Для этого необходимо использовать конструкцию `std::pair<node<K, T, D>*, node<K, T, D>*>`. Split так же работает рекурсивно. Он разделяет дерево T на два дерева L и R (которые являются возвращаемым значением) таким образом, что L содержит все элементы, меньшие по ключу X, а R содержит все элементы, большие X. Эта операция выполняется за O (log N). Также добавим функции поиска максимального и минимального узла. Очевидно, что минимальный элемент находится левее всех, а максимальный правее из определения и структуры дерева (бинарное дерево поиска по x). Код представлен в Код 6. Более подробный код приведен в Приложении.

```

1. std::pair<node<K, T, D>*, node<K, T, D>*> split(node<K, T, D>* cur, K x) {
2.     if (cur == NULL) return std::pair<node<K, T, D>*, node<K, T, D>*>(NULL,
3.         NULL);
4.     else if (x > cur->getkey()) {
5.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
6.             >getright(), x);
7.         ...
8.         return std::pair<node<K, T, D>*, node<K, T, D>*>(cur, tmp.second);
9.     } else {
10.        std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
11.            >getleft(), x);
12.        ...
13.        return std::pair<node<K, T, D>*, node<K, T, D>*>(tmp.first, cur);
14.    }
15. }

```

Код 6. Функция Split

Также можно сделать перегрузки основных операций для более широкого диапазона работы с деревьями. Например, добавление элемента с помощью Merge. Теперь, когда в классе есть главные операции, все остальные можно выразить через них. Операции удаления и добавления элемента, поиск максимального и минимального элемента. Вывод дерева осуществляется с помощью итераторов.

2.3 Декартово дерево по неявному ключу

Декартово дерево по неявному ключу наследуется от класса Treap с ключевым словом `protected`. Необходимо переопределить некоторые операции, а также добавить групповые функции. Например, Split. Можно заметить, что теперь в функции Split используются вспомогательные функции, такие как `get_size()`, `update()`. Они появляются в связи с отсутствием ключей у деревьев. Теперь мы используем индексы узлов. Код представлен в Код 7. Более подробный код приведен в Приложении.

```

1. std::pair<node<int, T, D>*, node<int, T, D>*> split(node<int, T, D>* cur,
2.     int x) {
3.     if (cur == NULL)
4.         return std::pair<node<int, T, D>*, node<int, T, D>*>(NULL, NULL);
5.     int l = (cur->getleft()) ? cur->getleft()->getsize() : 0;
6.     if (l >= x) {
7.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
8.             >getleft(), x); cur->setleft(tmp.second);
9.         ...
10.        update(cur); return std::pair<node<int, T, D>*, node<int, T, D>*>(tmp.f
11.            irst, cur);
12.    } else {
13.        std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
14.            >getright(), x - l - 1);
15.        ...
16.        update(cur); return std::pair<node<int, T, D>*, node<int, T, D>*>(cur,
17.            tmp.first);
18.    }
19. }

```



```
tmp.second);  
13. } }
```

Код 7. Переопределение функции Split

Здесь будут добавлены групповые функции и отложенные операции. Для демонстрации их работы введем функцию произведения и максимума на некотором отрезке. Также необходимо добавить операцию индексации узлов (функция будет обходить дерево и переустанавливать ключи узлов на основе их порядкового номера в обходе) и обновления узлов в дереве с помощью вычисления его размера и произведений значений узлов в его левом и правом поддеревьях.

2.4 Демонстрация работы в main()

Для демонстрации работы операций над деревьями, необходимо сначала создать и заполнить их. Далее запустим различные функции над ними и посмотрим на результат их работы. Код представлен в Код 8. Более подробный код приведен в Приложении.

```
1. treap<int, int, int> treap1;  
2. treap1.merge(1, 1, 1);  
3. ...  
4. treap<int, int, int> treap2;  
5. treap2.merge(1, 1, 1);  
6. ...  
7. treap2.add(8, 9, 1);  
8. treap2.deleteElement(6);  
9. std::cout << treap2 << " " << *treap2.getroot();  
10. imptreap<int, double> implictedtreap;  
11. implictedtreap.merge(1, 1.5);  
12. ...  
13. std::cout << implictedtreap << std::endl;  
14. imptreap<int, double> left, right;  
15. implictedtreap.itersplit(left, right, 5);
```

Код 8. Работа деревьев

Результаты выполнения функций приведены в Код 9, Код 10. Как можно видеть, операции выполняются верно.

```
Treap
( 1 1 1 )
( 2 5 1 )
( 3 2 1 )
( 4 7 1 )
( 5 3 1 )
( 6 6 1 )
( 7 4 1 )
root : ( 4 7 1 )

Treap after adding an element
( 1 1 1 )
( 2 5 1 )
( 3 2 1 )
( 4 7 1 )
( 5 3 1 )
( 6 6 1 )
( 7 4 1 )
( 8 9 1 )
root : ( 8 9 1 )

Treap after deleting an element
( 1 1 1 )
( 2 5 1 )
( 3 2 1 )
( 4 7 1 )
( 5 3 1 )
( 7 4 1 )
( 8 9 1 )
root : ( 8 9 1 )
```

Код 9. Вывод Treap

```
|||||||-----|||||||
ImpTreap:
( 1 1 1.5 )
( 2 2 5.5 )
( 3 3 2.5 )
( 4 4 7 )
( 5 5 3 )
( 6 6 6.001 )
( 7 7 4.1415 )

left:
( 1 1 1.5 )
( 2 2 5.5 )
( 3 3 2.5 )
( 4 4 7 )

right:
( 1 5 3 )
( 2 6 6.001 )
( 3 7 4.1415 )

original:
( 1 1 1.5 )
( 2 2 5.5 )
( 3 3 2.5 )
( 4 4 7 )
( 5 5 3 )
( 6 6 6.001 )
( 7 7 4.1415 )

size of right root: 3
new left:
( 1 1 1.5 )
( 2 2 5.5 )
( 3 3 2.5 )
( 4 4 7 )
( 5 5 3 )
( 6 6 6.001 )
( 7 7 4.1415 )

-----
product of left on [ 1 , 7 ]: 10764.5
```

Код 10. Вывод ImpTreap

Заключение

Декартово дерево неявное по ключу довольно удобная структура данных, позволяющая проводить быстрые операции над массивами данных. Несмотря на большой объем занимаемой памяти и сложность реализации эта структура представляет много возможностей. Например, быстрое выполнение операций на отрезке массива, над которым строится дерево. Также проще реализуется по сравнению, например, с настоящими самобалансирующимися деревьями вроде красно-чёрного. Сложность функции вычисления на отрезке стоит $O(\log n)$. Те же действия на обычном массиве стоили бы $O(n)$. В отличие от кучи и сбалансированного бинарного дерева поиска, декартово дерево работает не с отдельными элементами, а с отрезками массива, на которых рассчитывается значение функций. В общем декартово дерево по неявному ключу позволяет вставить элемент в любое место, переставить любой кусок массива куда угодно, совершать групповые операции с элементами. В групповые операции включается, естественно, и взятие функции от отрезка.

Декартово дерево, неявное по ключу, позволяет производить операции над массивом за логарифмическое время. Его следует использовать при наличии больших объемов памяти. При ее отсутствии лучше использовать дерево отрезков. Несмотря на более узкий круг операций, оно позволяет реализовать операции, не занимая много памяти. Самую простую реализацию и минимальные затраты по памяти имеет дерево Фенвика.

Также, используя идеи декартова дерева по неявному ключу, можно реализовать такую структуру данных как Rope.

Список литературы

1. Кормен, Т.Х. Алгоритмы: построение и анализ, 3-е издание / Т. Х. Кормен, Ч. И. Лейзерстон, Р.Л. Ривест, К. Штайн; пер. с англ. – Москва: «Вильямс», 2013 – 1328 с.
2. Вирт, Н. Алгоритмы и структуры данных / Н. Вирт; пер. с англ. – Санкт-Петербург, «Невский диалект», 2001 – 352 с.
3. Шилдт, Г. Самоучитель C++, 3-е издание / Г. Шилдт; пер. с англ. – Санкт-Петербург: БХВ-Петербург, 2003 – 688 с.
4. Шилдт, Г. Полный справочник по C++, 4-е издание / Г. Шилдт; пер. с англ. – Москва: Вильямс, 2006 – 800 с.
5. Шилдт, Г. Искусство программирования на C++ / Г. Шилдт; пер. с англ. – Санкт-Петербург: БХВ-Петербург, 2005 – 496 с.
6. Raimund Seidel, Cecilia R Ara. Randomized Search Trees – Foundations of Computer Science, University of California Berkeley, Berkeley, 1989.
7. Топп, У. Структуры данных в C++ / У. Топп, У. Форд; пер. с англ. – Москва: ЗАО «Издательство Бином», 1999 – 816 с.
8. Sleator, D.D. Self-adjusting binary search trees / D.D. Sleator, R.E. Tarjan // Journal of the ACM. – 1985 – 32 – pp. 652–686.

Приложение

```
1. #include<iostream>
2. #include<string>
3.
4. class MyExeption : std::exception {
5. protected:
6.     std::string str;
7. public:
8.     MyExeption(const std::string& str) {
9.         this->str = str;
10.    }
11.    virtual ~MyExeption() {}
12.    virtual void print_exeption() {
13.        std::cerr << str << std::endl;
14.    };
15.};
16.
17. class condition_exeption : MyExeption {
18. public:
19.     condition_exeption() : MyExeption("conditions do not allow to do this") {
20.    }
21.    virtual ~condition_exeption() { print_exeption(); }
22.};
23.
24. class size_exeption : MyExeption {
25. public:
26.     size_exeption() : MyExeption("wrong size or out of bounds") {}
27.    virtual ~size_exeption() { print_exeption(); }
28.};
29.
30. class nullptr_exeption : MyExeption {
31. public:
32.     nullptr_exeption() : MyExeption("pointer to 0") {}
33.    virtual ~nullptr_exeption() { print_exeption(); }
34.};
35.
36. class memory_exeption : MyExeption {
37. public:
38.     memory_exeption() : MyExeption("no memory allocated") {}
39.    virtual ~memory_exeption() { print_exeption(); }
40.};
41.
42. template<class K, class T, class D>
43. class node {
44. protected:
45.     K key;
46.     T value;
47.     int size;
48.     D product;
49.     node<K, T, D>* left;
50.     node<K, T, D>* right;
51.     node<K, T, D>* parent;
```

```

51. D data;
52. public:
53. node<K, T, D>() {
54.     key = K();
55.     value = T();
56.     left = right = parent = NULL;
57.     size = 1;
58.     product = T();
59.     data = D();
60. }
61.
62. node<K, T, D>(K key, T value, D data) {
63.     this->key = key, this->value = value;
64.     left = right = parent = NULL;
65.     size = 1;
66.     product = data;
67.     this->data = data;
68. }
69.
70. ~node<K, T, D>() {
71.     //std::cout << "exit ( " << key << " " << value << " )" << std::endl;
72. }
73. node<K, T, D>* setparent(node<K, T, D>* prnt) { return parent = prnt; }
74.
75. node<K, T, D>* setleft(node<K, T, D>* l) { return left = l; }
76.
77. node<K, T, D>* setright(node<K, T, D>* r) { return right = r; }
78.
79. node<K, T, D>* getparent() { return parent; }
80.
81. node<K, T, D>* getleft() { return left; }
82.
83. node<K, T, D>* getright() { return right; }
84.
85. K getkey() { return key; }
86.
87. T getvalue() { return value; }
88.
89. int getsize() { return size; }
90.
91. void setsize(int s) { size = s; }
92.
93. D getproduct() { return product; }
94.
95. void setproduct(D prod) { product = prod; }
96.
97. D getdata() { return data; }
98.
99. void setdata(D data) { this->data = data; }
100.
101.     void setkey(K newkey) { key = newkey; }
102.
103.     void setvalue(T newvalue) { value = newvalue; }

```

```

104.
105.     node<K, T, D>* succs() {
106.         if (right == NULL) {
107.             node<K, T, D>* tmp = this;
108.             while (true) {
109.                 if (tmp->parent == NULL)
110.                     return tmp->parent;
111.                 else if (tmp->parent->key > key) {
112.                     return tmp->parent;
113.                 }
114.                 else {
115.                     tmp = tmp->parent;
116.                 }
117.             }
118.         }
119.         else {
120.             node<K, T, D>* tmp = this->right;
121.             while (tmp->left != NULL) {
122.                 tmp = tmp->left;
123.             }
124.             return tmp;
125.         }
126.     }
127.
128.     friend std::ostream& operator<<(std::ostream& stream, node<K, T, D>
    & node) {
129.         stream << "(" << node.key << " " << node.value << " " << node.dat
    a << " )" << std::endl;
130.         return stream;
131.     }
132.
133.     friend std::ostream& operator<<(std::ostream& stream, const node<K,
    T, D>& node) {
134.         stream << "(" << node.key << " " << node.value << " " << node.dat
    a << " )" << std::endl;
135.         return stream;
136.     }
137. };
138.
139. template<class K, class T, class D>
140. class treapiterator {
141. protected:
142.     node<K, T, D>* cur;
143. public:
144.     treapiterator<K, T, D>() {
145.         cur = NULL;
146.     }
147.     treapiterator<K, T, D>(node<K, T, D>* ptr) {
148.         cur = ptr;
149.     }
150.     treapiterator<K, T, D>& operator++(int v) {
151.         cur = cur->succs();
152.         return *this;

```

```

153.     }
154.
155.     treapiterator<K, T, D>& operator++(int v) const {
156.         cur = cur->sucs();
157.         return treapiterator<K, T, D>(cur);
158.     }
159.     treapiterator<K, T, D>& operator++() {
160.         cur = cur->sucs();
161.         return treapiterator<K, T, D>(cur);
162.     }
163.     treapiterator<K, T, D>& operator++() const {
164.         cur = cur->sucs();
165.         return treapiterator<K, T, D>(cur);
166.     }
167.     treapiterator<K, T, D>& operator=(node<K, T, D>* ptr) {
168.         cur = ptr;
169.         return *this;
170.     }
171.     treapiterator<K, T, D>& operator=(const node<K, T, D>* ptr) const {
172.         cur = ptr;
173.         return *this;
174.     }
175.     treapiterator<K, T, D>& operator=(const treapiterator<K, T, D>& ite
r) {
176.         cur = iter.cur;
177.         return *this;
178.     }
179.     treapiterator<K, T, D>& operator=(const treapiterator<K, T, D>& ite
r) const {
180.         cur = iter.cur;
181.         return *this;
182.     }
183.
184.     bool operator==(const node<K, T, D>* ptr) {
185.         return cur == ptr;
186.     }
187.     bool operator==(const node<K, T, D>* ptr) const {
188.         return cur == ptr;
189.     }
190.     bool operator!=(const node<K, T, D>* ptr) {
191.         return cur != ptr;
192.     }
193.     bool operator!=(const node<K, T, D>* ptr) const {
194.         return cur != ptr;
195.     }
196.
197.     bool operator==(const treapiterator<K, T, D>& ptr) {
198.         return cur == ptr.cur;
199.     }
200.     bool operator==(const treapiterator<K, T, D>& ptr) const {
201.         return cur == ptr.cur;
202.     }
203.     bool operator!=(const treapiterator<K, T, D>& ptr) {

```



```

204.         return cur != ptr.cur;
205.     }
206.     bool operator!=(const treapiterator<K, T, D>& ptr) const {
207.         return cur != ptr.cur;
208.     }
209.
210.     node<K, T, D>* operator*() {
211.         return cur;
212.     }
213.     node<K, T, D>* operator*() const {
214.         return cur;
215.     }
216. };
217.
218. template<class K, class T, class D>
219. class treap {
220. protected:
221.     node<K, T, D>* root;
222.
223.     //2)
224.     node<K, T, D>* merge2(const node<K, T, D>* L, const node<K, T, D>*
225. R) {
226.         if (L == NULL || R == NULL) return (L) ? new node<K, T, D>(*L) : n
227. ew node<K, T, D>(*R);
228.         if (L->getvalue() > R->getvalue()) {
229.             if (L->getkey() < R->getkey()) {
230.                 node<K, T, D>* tmp = merge2(L->getright(), R);
231.                 if (tmp != NULL)
232.                     tmp->setparent(new node<K, T, D>(* L));
233.                 L->setright(tmp);
234.             }
235.             else {
236.                 node<K, T, D>* tmp = merge2(L->getleft(), R);
237.                 if (tmp != NULL)
238.                     tmp->setparent(new node<K, T, D>(* L));
239.                 L->setleft(tmp);
240.             }
241.             return new node<K, T, D>(*L);
242.         }
243.         else {
244.             if (L->getkey() < R->getkey()) {
245.                 node<K, T, D>* tmp = merge2(L, R->getleft());
246.                 if (tmp != NULL)
247.                     tmp->setparent(new node<K, T, D>(* R));
248.                 R->setleft(tmp);
249.             }
250.             else {
251.                 node<K, T, D>* tmp = merge2(L, R->getright());
252.                 if (tmp != NULL)
253.                     tmp->setparent(new node<K, T, D>(* R));
254.                 R->setright(tmp);
255.             }

```

```

255.         return new node<K, T, D>(* R);
256.     }
257. }
258.
259. //1)
260. node<K, T, D>* merge(node<K, T, D>* L, node<K, T, D>* R) {
261.     if (L == NULL || R == NULL) return (L) ? L : R;
262.
263.     if (L->getvalue() > R->getvalue()) {
264.         if (L->getkey() < R->getkey()) {
265.             node<K, T, D>* tmp = merge(L->getright(), R);
266.             if (tmp != NULL)
267.                 tmp->setparent(L);
268.             L->setright(tmp);
269.         }
270.         else {
271.             node<K, T, D>* tmp = merge(L->getleft(), R);
272.             if (tmp != NULL)
273.                 tmp->setparent(L);
274.             L->setleft(tmp);
275.         }
276.         return L;
277.     }
278.     else {
279.         if (L->getkey() < R->getkey()) {
280.             node<K, T, D>* tmp = merge(L, R->getleft());
281.             if (tmp != NULL)
282.                 tmp->setparent(R);
283.             R->setleft(tmp);
284.         }
285.         else {
286.             node<K, T, D>* tmp = merge(L, R->getright());
287.             if (tmp != NULL)
288.                 tmp->setparent(R);
289.             R->setright(tmp);
290.         }
291.         return R;
292.     }
293. }
294.
295. //1)
296. std::pair<node<K, T, D>*, node<K, T, D>*> split(node<K, T, D>* cur,
    K x) {
297.     if (cur == NULL) return std::pair<node<K, T, D>*, node<K, T, D>*>(
        NULL, NULL);
298.     else if (x > cur->getkey()) {
299.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
            >getright(), x);
300.         cur->setright(tmp.first);
301.         if (tmp.first != NULL)
302.             tmp.first->setparent(cur);
303.         if (tmp.second != NULL)
304.             tmp.second->setparent(NULL);

```

```

305.         return std::pair<node<K, T, D>*, node<K, T, D>*>(cur, tmp.second)
306.     };
307.     else {
308.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(cur-
>getleft(), x);
309.         cur->setleft(tmp.second);
310.         if (tmp.second != NULL)
311.             tmp.second->setparent(cur);
312.         if (tmp.first != NULL)
313.             tmp.first->setparent(NULL);
314.         return std::pair<node<K, T, D>*, node<K, T, D>*>(tmp.first, cur);
315.     }
316. }
317.
318. void cutnode(node<K, T, D>* n) {
319.     n->setparent(NULL);
320.     n->setleft(NULL);
321.     n->setright(NULL);
322. }
323.
324. void free(node<K, T, D>* t) {
325.     if (t == NULL) return;
326.     free(t->getleft());
327.     free(t->getright());
328.     delete t;
329. }
330.
331. public:
332.     treap<K, T, D>() {
333.         root = NULL;
334.     }
335.
336.     treap<K, T, D>(node<K, T, D>* newroot) {
337.         root = newroot;
338.     }
339.
340.     ~treap<K, T, D>() {
341.         free(root);
342.     }
343.     node<K, T, D>* merge(node<K, T, D>* R) {
344.         root = merge(root, R);
345.         return root;
346.     }
347.
348.     node<K, T, D>* merge(treap<K, T, D>& R) {
349.         root = merge(root, R.root);
350.         R.root = NULL;
351.         return root;
352.     }
353.
354.     //1)
355.     node<K, T, D>* merge(K key, T value, D data) {

```

```

356.     node<K, T, D>* tmp = new node<K, T, D>(key, value, data);
357.     return root = merge(root, tmp);
358. }
359.
360.     //добавление элемента
361.     node<K, T, D>* add(K key, T value, D data) {
362.         treap<K, T, D> L;
363.         treap<K, T, D> R;
364.         itersplit(L, R, key);
365.         L.merge(key, value, data);
366.         L.merge(R);
367.         std::swap(root, L.root);
368.         return root;
369.     }
370.
371.     //удаление элемента i hope... YES IT'S WORKING!
372.     node<K, T, D>* deleteElement(K key) {
373.         node<K, T, D> *less, *equal, *greater;
374.         less = split(getroot(), key).first;
375.         greater = split(getroot(), key).second;
376.
377.         equal = split(greater, key + 1).first;
378.         greater = split(greater, key + 1).second;
379.
380.         root = merge(less, greater);
381.         return root;
382.     }
383.
384.     //обычное разделение
385.     void split(treap<K, T, D>& L, treap<K, T, D>& R, K x) {
386.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
387.         std::pair<node<K, T, D>*, node<K, T, D>*> tmp = split(root, x);
388.         L.root = tmp.first;
389.         R.root = tmp.second;
390.         root = NULL;
391.     }
392.
393.     //разделение через итераторы
394.     void itersplit(treap<K, T, D>& L, treap<K, T, D>& R, K x) {
395.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
396.         for (treapiterator<K, T, D> iter = min(); iter != NULL; iter++) {
397.             if ((*iter)->getkey() < x) {
398.                 L.merge((*iter)->getkey(), (*iter)->getvalue(), (*iter)-
>getdata());
399.             }
400.             else {
401.                 R.merge((*iter)->getkey(), (*iter)->getvalue(), (*iter)-
>getdata());
402.             }
403.         }
404.     }
405.
406.     node<K, T, D>* min() {

```

```

407.     if (root == NULL) return NULL;
408.     node<K, T, D>* tmp = root;
409.     while (tmp->getleft() != NULL) {
410.         tmp = tmp->getleft();
411.     }
412.     return tmp;
413. }
414.
415. node<K, T, D>* max() {
416.     if (root == NULL) return NULL;
417.     node<K, T, D>* tmp = root;
418.     while (tmp->getright() != NULL) {
419.         tmp = tmp->getright();
420.     }
421.     return tmp;
422. }
423.
424. node<K, T, D>* getroot() {
425.     return root;
426. }
427.
428. friend std::ostream& operator<<(std::ostream& stream, treap<K, T, D
    >& treap) {
429.     for (treapiterator<K, T, D> iter = treap.min(); iter != NULL; iter
        ++) {
430.         stream << **iter;
431.     }
432.     return stream;
433. }
434.
435. friend std::ostream& operator<<(std::ostream& stream, const treap<K
    , T, D>& treap) {
436.     for (treapiterator<K, T, D> iter = treap.min(); iter != NULL; iter
        ++) {
437.         stream << **iter;
438.     }
439.     return stream;
440. }
441. };
442.
443. template<class T, class D>
444. class imptreap : protected treap<int, T, D> {
445. protected:
446.
447.     node<int, T, D>* merge(node<int, T, D>* L, node<int, T, D>* R) {
448.         if (L == NULL || R == NULL) return (L) ? L : R;
449.
450.         if (L->getvalue() > R->getvalue()) {
451.
452.             node<int, T, D>* tmp = merge(L->getright(), R);
453.             if (tmp != NULL)
454.                 tmp->setparent(L);
455.             L->setright(tmp);

```

```

456.     update(L);
457.     return L;
458. }
459. else {
460.     node<int, T, D>* tmp = merge(L, R->getleft());
461.     if (tmp != NULL)
462.         tmp->setparent(R);
463.     R->setleft(tmp);
464.     update(R);
465.     return R;
466. }
467. }
468.
469.     //обход дерева и переустановка ключей узлов на основе их порядковог
о номера в обходе
470.     int fix_keys(node<int, T, D>* ptr, int x) {
471.         if (ptr == NULL) return x;
472.         int tmp = fix_keys(ptr->getleft(), x);
473.         ptr->setkey(tmp);
474.         return fix_keys(ptr->getright(), tmp + 1);
475.     }
476.
477.     //обновление узлов в дереве, с помощью вычисления его размеров и п
роизведений значений узлов в его левом и правом поддеревьях
478.     void update(node<int, T, D>* n) {
479.         if (n == NULL) return;
480.         n->setsize(1 + ((n->getleft()) ? n->getleft()-
>getsize() : 0) + ((n->getright()) ? n->getright()->getsize() : 0));
481.         n->setproduct(n->getdata() * ((n->getleft()) ? n->getleft()-
>getproduct() : 1) * ((n->getright()) ? n->getright()->getproduct() : 1));
482.     }
483.
484.     std::pair<node<int, T, D>*, node<int, T, D>*> split(node<int, T, D>
* cur, int x) {
485.         if (cur == NULL) {
486.             return std::pair<node<int, T, D>*, node<int, T, D>*>(NULL, NULL);
487.         }
488.         int l = (cur->getleft()) ? cur->getleft()->getsize() : 0;
489.         if (l >= x) {
490.             std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
>getleft(), x);
491.             cur->setleft(tmp.second);
492.             if (tmp.second != NULL)
493.                 tmp.second->setparent(cur);
494.             if (tmp.first != NULL)
495.                 tmp.first->setparent(NULL);
496.             update(cur);
497.             return std::pair<node<int, T, D>*, node<int, T, D>*>(tmp.first, c
ur);
498.         }
499.         else {
500.             std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(cur-
>getright(), x - l - 1);

```

```

501.         cur->setright(tmp.first);
502.         if (tmp.second != NULL)
503.             tmp.second->setparent(NULL);
504.         if (tmp.first != NULL)
505.             tmp.first->setparent(cur);
506.         update(cur);
507.         return std::pair<node<int, T, D>*, node<int, T, D>*>(cur, tmp.second);
508.     }
509. }
510. public:
511.
512.     imptreap<T, D>() :treap<int, T, D>() {}
513.     imptreap<T, D>(node<int, T, D>* ptr) : treap<int, T, D>(ptr) {}
514.     ~imptreap<T, D>() {}
515.     node<int, T, D>* merge(imptreap<T, D>& R) {
516.         treap<int, T, D>::root = merge(treap<int, T, D>::root, R.root);
517.         R.root = NULL;
518.         return treap<int, T, D>::root;
519.     }
520.     node<int, T, D>* merge(T value, D data) {
521.         node<int, T, D>* tmp = new node<int, T, D>(1, value, data);
522.         return treap<int, T, D>::root = merge(treap<int, T, D>::root, tmp);
523.     }
524.
525.     void split(imptreap<T, D>& L, imptreap<T, D>& R, int x) {
526.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
527.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp = split(treap<int, T, D>::root, x - 1);
528.         L.root = tmp.first;
529.         R.root = tmp.second;
530.         treap<int, T, D>::root = NULL;
531.     }
532.
533.     void itersplit(imptreap<T, D>& L, imptreap<T, D>& R, int x) {
534.         if (L.root != NULL || R.root != NULL) throw condition_exeption();
535.         for (treapiterator<int, T, D> iter = min(); iter != NULL; iter++)
536.         {
537.             if ((*iter)->getkey() < x) {
538.                 L.merge((*iter)->getvalue(), (*iter)->getdata());
539.             }
540.             else {
541.                 R.merge((*iter)->getvalue(), (*iter)->getdata());
542.             }
543.         }
544.
545.         //произведение
546.         D find_product(int start, int end) {
547.             if (start > end || end < 0 || start < 0 || end > treap<int, T, D>::root->getsize())
548.                 || start > treap<int, T, D>::root->getsize())

```

```

549.         throw condition_exeption();
550.
551.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp1 = split(treap<int, T, D>::root, start - 1);
552.         std::pair<node<int, T, D>*, node<int, T, D>*> tmp2 = split(tmp1.second, end - start + 1);
553.         D tmp = tmp2.first->getproduct();
554.         merge(tmp2.first, tmp2.second);
555.         merge(tmp1.first, tmp1.second);
556.         return tmp;
557.     }
558.
559.     //групповая функция
560.     int Maximum(int l, int r)
561.     {
562.         std::pair<node<int, T, D>*, node<int, T, D>*> p1;
563.         std::pair<node<int, T, D>*, node<int, T, D>*> p2;
564.         node<int, T, D>* QWERTY = getroot();
565.         p1 = split(QWERTY, r);
566.         p2 = split(p1.first, l);
567.         int res = p2.second->getproduct();
568.         QWERTY = merge(p2.first, merge(p2.second, p1.second));
569.         return res;
570.     }
571.
572.     int size(node<int, T, D>* ptr) {
573.         return ptr->getsize();
574.     }
575.
576.     node<int, T, D>* getroot() {
577.         return treap<int, T, D>::root;
578.     }
579.
580.     node<int, T, D>* min() {
581.         if (treap<int, T, D>::root == NULL) return NULL;
582.         node<int, T, D>* tmp = treap<int, T, D>::root;
583.         while (tmp->getleft() != NULL) {
584.             tmp = tmp->getleft();
585.         }
586.         return tmp;
587.     }
588.     node<int, T, D>* max() {
589.         if (treap<int, T, D>::root == NULL) return NULL;
590.         node<int, T, D>* tmp = treap<int, T, D>::root;
591.         while (tmp->getright() != NULL) {
592.             tmp = tmp->getright();
593.         }
594.         return tmp;
595.     }
596.     friend std::ostream& operator<<(std::ostream& stream, imptreap<T, D>& treap) {
597.         treap.fix_keys(treap.root, 1);

```



```

598.         for (treapiterator<int, T, D> iter = treap.min(); iter != NULL; it
           er++) {
599.             stream << **iter;
600.         }
601.         return stream;
602.     }
603.     friend std::ostream& operator<<(std::ostream& stream, const imptrea
           p<T, D>& treap) {
604.         treap.fix_keys(treap.root, 1);
605.         for (treapiterator<int, T, D> iter = treap.min(); iter != NULL; it
           er++) {
606.             stream << **iter;
607.         }
608.         return stream;
609.     }
610. };
611.
612.
613.     int main() {
614.         treap<int, int, int> treap1;
615.         treap1.merge(1, 1, 1);
616.         treap1.merge(2, 5, 1);
617.         treap1.merge(3, 2, 1);
618.         treap1.merge(4, 7, 1);
619.         treap1.merge(5, 3, 1);
620.         treap1.merge(6, 6, 1);
621.         treap1.merge(7, 4, 1);
622.
623.         treap<int, int, int> treap2;
624.         treap2.merge(1, 1, 1);
625.         treap2.merge(2, 5, 1);
626.         treap2.merge(3, 2, 1);
627.         treap2.merge(4, 7, 1);
628.         treap2.merge(5, 3, 1);
629.         treap2.merge(6, 6, 1);
630.         treap2.merge(7, 4, 1);
631.
632.         std::cout << "Treap\n" << treap2 << " root : " << *treap2.getroot()
           ;
633.         treap2.add(8, 9, 1);
634.         std::cout << "\nTreap after adding an element\n" << treap2 << " roo
           t : " << *treap2.getroot();
635.         treap2.deleteElement(6);
636.         std::cout << "\nTreap after deleting an element\n" << treap2 << " r
           oot : " << *treap2.getroot();
637.
638.         std::cout << "\n";
639.
640.         //std::cout << treap1 << " " << *treap1.getroot();
641.         std::cout << treap2 << " " << *treap2.getroot();
642.         std::cout << std::endl << " |-----
           -----| " << std::endl;
643.

```

```

644.      //treap1.merge(8, 9, 1);
645.
646.      //treap<int, int, int> treap3 = merge2(*treap1.getroot(), *treap2.g
        etroot());
647.      //treap1.merge2(treap1.getroot(), );
648.      //treap1.deleteElement(treap1.getroot(), 5);
649.
650.      std::cout << treap1 << " " << *treap1.getroot();
651.
652.      std::cout << std::endl << "|-----
        -----|" << std::endl;
653.
654.      std::cout << std::endl << "-----
        -----" << std::endl;
655.
656.      treap<int, int, int> left1;
657.      treap<int, int, int> right1;
658.      treap1.split(left1, right1, 5);
659.      std::cout << left1 << std::endl << right1 << std::endl;
660.
661.      std::cout << std::endl << "-----
        -----" << std::endl;
662.      right1.merge(left1);
663.      std::cout << right1;
664.
665.      std::cout << std::endl << "=====
        =====" << std::endl;
666.
667.      treap<int, int, int> left2;
668.      treap<int, int, int> right2;
669.      treap2.itorsplit(left2, right2, 5);
670.      std::cout << left2 << std::endl << right2 << std::endl;
671.
672.      std::cout << std::endl << "-----
        -----" << std::endl;
673.      right2.merge(left2);
674.      std::cout << right2;
675.
676.      std::cout << std::endl << "|||||
        |||||" << std::endl;
677.      imptreap<int, double> implicatedtreap;
678.      implicatedtreap.merge(1, 1.5);
679.      implicatedtreap.merge(2, 5.5);
680.      implicatedtreap.merge(3, 2.5);
681.      implicatedtreap.merge(4, 7.0);
682.      implicatedtreap.merge(5, 3.0);
683.      implicatedtreap.merge(6, 6.001);
684.      implicatedtreap.merge(7, 4.1415);
685.
686.      std::cout << "ImpTreap:\n" << implicatedtreap << std::endl;
687.
688.      imptreap<int, double> left, right;
689.      implicatedtreap.itorsplit(left, right, 5);

```

```

690.     std::cout << "left:\n" << left << std::endl << "right:\n" << right
    << std::endl << "original:\n" << implicatedtreap << std::endl;
691.
692.     std::cout << "size of right root: " << right.size(right.getroot())
    << std::endl;
693.     left.merge(right);
694.     std::cout << "new left:\n" << left << std::endl;
695.     std::cout << std::endl << "-----
    -----" << std::endl;
696.     int a = 1, b = 7;
697.     std::cout << "product of left on [ " << a << " , " << b << " ]: " <
    < left.find_product(a, b) << std::endl;
698.
699.     std::cout << std::endl << "-----
    -----" << std::endl;
700.     treap<int, int, int> tr;
701.     for (int i = 0; i < 15; i++) {
702.         tr.add(i, rand() % 100, i);
703.     }
704.     std::cout << tr;
705.
706.     return 0;
707. }

```